

Ch13: Selection Sort

Slide 1: Understanding Selection Sort

Selection Sort is a sorting method that repeatedly looks through the **unsorted part of an array**, finds the smallest element, and places it at the beginning of that unsorted section.

Think of arranging books from **shortest to tallest**. You look through all the remaining books, find the shortest one, place it first, and then repeat the process with the books that remain.

Suppose we have:

5 3 8 2 7

The first position should contain the smallest value.

So we inspect the entire array:

5 3 8 2 7
 ↑
 smallest

The smallest value is:

2

Move 2 to the first position:

2 3 8 5 7

Now the first position is correct.

The process continues with the remaining part:

[2] | 3 8 5 7
 ↑
 unsorted

Selection Sort in One Sentence

Find the smallest value, place it where it belongs, and repeat.

The important part is that Selection Sort does not immediately move every value around. It first **selects** the element that belongs at the current position.

Key idea:

Selection Sort builds the sorted array one position at a time by selecting the smallest remaining element.

Slide 2: Finding the Minimum Element

Let's trace Selection Sort carefully.

Start with:

5 3 8 2

We want the smallest value to go into position 0.

So we examine each value:

5 → current smallest
3 → smaller than 5
8 → larger than 3
2 → smaller than 3

Therefore:

smallest = 2

The array currently looks like:

5 3 8 2
↑ ↑
first minimum
position

We then exchange the first element with the minimum:

5 3 8 2
↓ ↓
2 3 8 5

Result:

2 3 8 5

The first position is now permanently sorted.

What Happened Internally?

Selection Sort keeps track of the **position** of the smallest value.

For example:

5 3 8 2

↑
i

Initially, we assume the first element is the minimum:

```
int min = i;
```

Then we compare the remaining elements:

```
if (arr[j] < arr[min])  
{  
    min = j;  
}
```

Notice that `min` stores an **index**, not the actual minimum value.

For example:

```
arr = 5 3 8 2
```

```
min = 0
```

Then:

```
arr[0] = 5
```

When we find 3:

```
3 < 5
```

so:

```
min = 1
```

When we find 2:

$2 < 3$

so:

min = 3

At the end:

min = 3

That tells us:

The smallest element is at index 3.

Key idea:

Selection Sort remembers **where** the smallest element is, then swaps that element into the correct position.

Slide 3: The Sorted and Unsorted Parts

One of the easiest ways to understand Selection Sort is to imagine the array divided into two sections:

[SORTED] | [UNSORTED]

The sorted section grows from **left to right**.

Consider:

5 3 8 2 7 1

Pass 1

Find the smallest value in the entire array:

5 3 8 2 7 1
 ↑
 minimum

Minimum:

1

Place it at the beginning:

1 | 3 8 2 7 5

Now:

[1] | [3 8 2 7 5]

↑ ↑
sorted unsorted

Pass 2

The first position is already sorted.

Now search only:

3 8 2 7 5

The minimum is:

2

Place it at the beginning of the unsorted section:

1 2 | 8 3 7 5

Pass 3

Search:

8 3 7 5

Minimum:

3

Result:

1 2 3 | 8 7 5

Pass 4

Search:

8 7 5

Minimum:

5

Result:

1 2 3 5 | 7 8

Finally:

1 2 3 5 7 8

The process can be visualized as:

Start:

5 3 8 2 7 1

Pass 1:

[1] | 3 8 2 7 5

Pass 2:

[1 2] | 8 3 7 5

Pass 3:

[1 2 3] | 8 7 5

Pass 4:

[1 2 3 5] | 7 8

Done:

[1 2 3 5 7 8]

Notice what happens after every pass:

Sorted section grows

↓

[1] | ...

[1 2] | ...

[1 2 3] | ...

[1 2 3 5] | ...

That growing sorted section is the central idea behind Selection Sort.

Key idea:

After every pass, one more element is placed permanently into the sorted portion.

Slide 4: Selection Sort in C

The basic Selection Sort structure is:

```
for (int i = 0; i < n - 1; i++)
{
    int min = i;

    for (int j = i + 1; j < n; j++)
    {
        if (arr[j] < arr[min])
        {
            min = j;
        }
    }

    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
```

Let's understand each part.

Step 1: Choose the Position

```
for (int i = 0; i < n - 1; i++)
```

`i` represents the position where the next smallest element should be placed.

For example:

`i = 0` → find smallest for position 0

`i = 1` → find smallest for position 1

`i = 2` → find smallest for position 2

Step 2: Assume the Current Element Is Minimum

```
int min = i;
```

We initially assume:

```
arr[i] = smallest
```

Then we search to see whether something smaller exists.

Step 3: Search the Unsorted Part

```
for (int j = i + 1; j < n; j++)
```

The search starts at `i + 1` because `arr[i]` is already our current minimum candidate.

For example, if:

`i = 2`

then:

[sorted]	[current]	[unsorted]
0 1 2	3 4 5	

So `j` begins at:

`j = 3`

Step 4: Find a Smaller Element

```
if (arr[j] < arr[min])  
{  
    min = j;  
}
```

If a smaller element is found, we remember its index.

Step 5: Swap

After searching the entire unsorted section:

```
int temp = arr[i];  
arr[i] = arr[min];  
arr[min] = temp;
```

This places the smallest element at position `i`.

Complete Flow

Choose position `i`



Assume `arr[i]` is minimum



Search remaining elements



Find smaller element?



Remember its index

↓
Finish searching
↓
Swap arr[i] with arr[min]
↓
Sorted section grows
↓
Move to next position

The code is therefore not a magical block of punctuation humans are expected to memorize. Each part has one job.

Key idea:

The outer loop chooses the position. The inner loop finds the minimum. The swap places it correctly.

Slide 5: Selection Sort in Action

Selection Sort becomes much clearer when we follow the elements through several passes.

Consider:

6 3 8 2 7 1 5 4

Pass 1

Search the entire array.

6 3 8 2 7 1 5 4
↑
minimum = 1

Swap 6 and 1:

1 3 8 2 7 6 5 4

Sorted section:

[1] | 3 8 2 7 6 5 4

Pass 2

Search the remaining values:

3 8 2 7 6 5 4

↑
minimum = 2

Swap 3 and 2:

1 2 8 3 7 6 5 4

Sorted section:

[1 2] | 8 3 7 6 5 4

Pass 3

Search:

8 3 7 6 5 4

Minimum:

3

After swapping:

1 2 3 8 7 6 5 4

Pass 4

Search:

8 7 6 5 4

Minimum:

4

After swapping:

1 2 3 4 7 6 5 8

The sorted portion continues growing:

[1] | 3 8 2 7 6 5 4

[1 2] | 8 3 7 6 5 4

[1 2 3] | 8 7 6 5 4

[1 2 3 4] | 7 6 5 8

The same process continues until the entire array is sorted.

Selection Sort Mental Model

Selection Sort



Find smallest



Remember its index



Swap with first unsorted position



Sorted section grows



Repeat

Core Takeaway

Selection Sort repeatedly selects the smallest element from the unsorted section and places it at the beginning of that section.

For ascending Selection Sort:

Find minimum → Swap → Move right → Repeat

For descending Selection Sort, the same overall structure can be used, but instead of finding the **minimum**, we find the **maximum**:

Find maximum → Swap → Move right → Repeat

That small change is the recurring theme of sorting: the machinery stays remarkably similar, while the comparison decides what the algorithm considers “correct.”